

Algoritmos evolutivos

Búsqueda y optimización inspiradas en la evolución

Joaquín F. Sánchez

Introducción a la Inteligencia Artificial

Del espacio de búsqueda a una población de soluciones

Al finalizar la sesión, el estudiante podrá:

- ① explicar por qué un algoritmo evolutivo es una técnica de búsqueda estocástica;
- ② representar una solución mediante cromosomas y genes;
- ③ interpretar selección, cruce, mutación y reemplazo;
- ④ diseñar una función de aptitud con restricciones;
- ⑤ reconocer los compromisos entre exploración, explotación y costo computacional.

Si no podemos evaluar todas las soluciones posibles,
¿cómo podemos encontrar una solución suficientemente
buena?

Idea central

Mantener varias soluciones candidatas, favorecer las más aptas y producir nuevas variantes durante sucesivas generaciones.

De la evolución biológica a la computación

Principios de inspiración

- ▶ Variación entre individuos.
- ▶ Herencia de características.
- ▶ Selección según adaptación.
- ▶ Reproducción con cambios.

Traducción algorítmica

- ▶ Diversidad de soluciones.
- ▶ Combinación de genes.
- ▶ Aptitud o desempeño.
- ▶ Cruce y mutación.

Precisión importante

Es una metáfora computacional: el objetivo es optimizar, no reproducir toda la complejidad de la evolución natural.

¿Qué es un algoritmo evolutivo?

Definición de trabajo

Método de búsqueda poblacional y estocástico que mejora soluciones candidatas mediante evaluación, selección y variación.

Resulta útil cuando

- ▶ el espacio de búsqueda es grande;
- ▶ hay muchas combinaciones válidas;
- ▶ la función objetivo es compleja;
- ▶ una buena aproximación es aceptable.

No garantiza

- ▶ hallar siempre el óptimo global;
- ▶ repetir exactamente el resultado;
- ▶ superar métodos exactos en todo problema;
- ▶ corregir una mala representación.

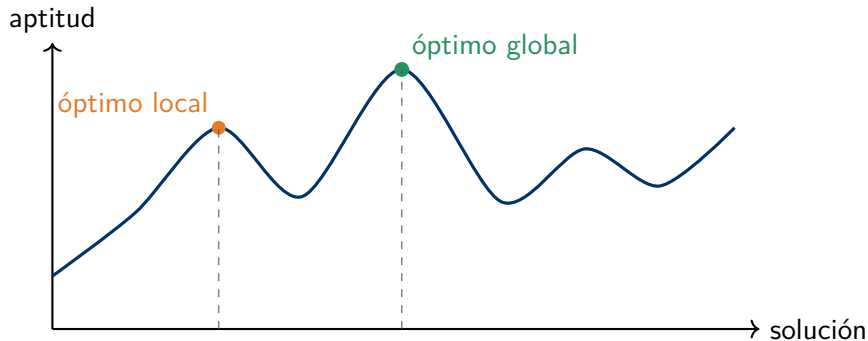
Familia de métodos evolutivos

- ▶ **Algoritmos genéticos:** cromosomas, cruce y mutación.
- ▶ **Estrategias evolutivas:** énfasis en variables reales y mutación.
- ▶ **Programación evolutiva:** evolución de comportamientos.
- ▶ **Programación genética:** evolución de expresiones o programas.
- ▶ **Evolución diferencial:** diferencias vectoriales para generar candidatos.

En esta sesión

Nos concentraremos en el **algoritmo genético**, el caso desarrollado en el capítulo.

Óptimo local y óptimo global



La diversidad inicial y la mutación reducen el riesgo de converger demasiado pronto en una región inferior.

Problema de la mochila

Seleccionar objetos para maximizar el valor total sin exceder la capacidad:

$$\max \sum_{i=1}^n v_i x_i \quad \text{sujeto a} \quad \sum_{i=1}^n w_i x_i \leq C$$
$$x_i \in \{0, 1\}$$

Interpretación

- ▶ v_i : valor del objeto i .
- ▶ w_i : peso del objeto i .
- ▶ C : capacidad máxima.
- ▶ x_i : incluir o excluir.

Ejemplo pequeño: capacidad de 9 kg

ID	Objeto	Peso	Valor
1	Perlas	3	4
2	Oro	7	7
3	Corona	4	5
4	Moneda	1	1
5	Hacha	5	4
6	Espada	4	3
7	Anillo	2	5
8	Copa	3	1

El cromosoma $\mathbf{x} = 10100010$ elige perlas, corona y anillo: peso 9, valor 14.

¿Por qué no probar todas las opciones?

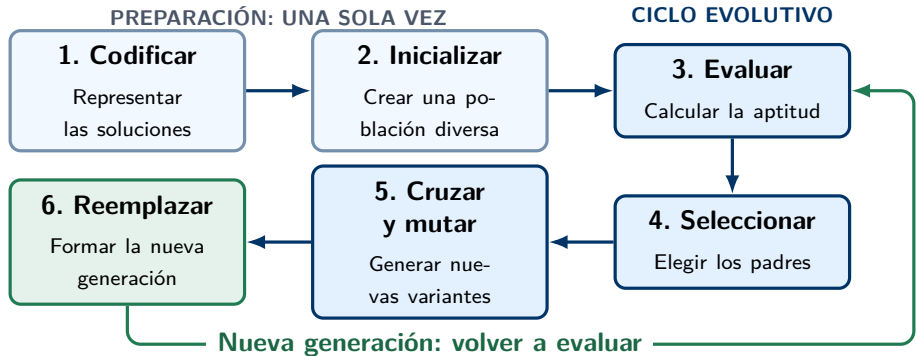
Con n decisiones binarias existen 2^n combinaciones, incluida la mochila vacía.

n	Combinaciones	Escala
8	256	manejable
26	67 108 864	costosa
50	1.13×10^{15}	impracticable por fuerza bruta

Cambio de objetivo

El algoritmo genético intercambia la garantía de optimalidad por una búsqueda eficiente de soluciones de alta calidad.

Ciclo de vida



El ciclo se repite hasta cumplir una **condición de parada**.

1. Codificación de soluciones

Vocabulario

- ▶ **Cromosoma:** una solución candidata completa.
- ▶ **Gen:** una posición o variable de la solución.
- ▶ **Alelo:** el valor concreto de un gen.
- ▶ **Genotipo:** representación codificada.
- ▶ **Fenotipo:** solución interpretada en el problema.

1 0 1 0 0 0 1 0

cromosoma binario de 8 genes

Elegir una codificación adecuada

Codificación	Ejemplo	Problemas típicos
Binaria	101001	selección de elementos
Real	(1.2, -0.8, 4.5)	ajuste de parámetros
Permutación	(3, 1, 5, 2, 4)	rutas y secuencias
Árbol	expresión jerárquica	programas y reglas

Criterio

Los operadores deben producir soluciones interpretables y, de ser posible, válidas para el problema.

2. Crear la población inicial

Objetivo: cubrir regiones distintas del espacio de búsqueda.

- ▶ Inicialización aleatoria.
- ▶ Tamaño fijo o dinámico.
- ▶ Validación de restricciones.
- ▶ Evitar sesgos de generación.

Individuo	Cromosoma
<i>A</i>	10100010
<i>B</i>	01011000
<i>C</i>	00100111
<i>D</i>	11000001

Una población pequeña reduce costo; una población mayor suele aportar más diversidad.

3. Función de aptitud

Para la mochila, una función simple es:

$$f(\mathbf{x}) = \begin{cases} \sum_i v_i x_i, & \text{si } \sum_i w_i x_i \leq C, \\ 0, & \text{si excede la capacidad.} \end{cases}$$

Debe reflejar

El objetivo que realmente se desea optimizar.

Riesgo

Una función mal formulada conduce al algoritmo hacia soluciones equivocadas.

Las restricciones también pueden manejarse mediante penalizaciones graduales o reparación.

Ejercicio rápido de aptitud

Capacidad $C = 9$ kg. Evalúe los cromosomas:

Cromosoma	Objetos	Peso	Aptitud
10100010	1, 3, 7	$3 + 4 + 2 = 9$	
11000000	1, 2	$3 + 7 = 10$	
00010010	4, 7	$1 + 2 = 3$	

Ejercicio rápido de aptitud

Capacidad $C = 9$ kg. Evalúe los cromosomas:

Cromosoma	Objetos	Peso	Aptitud
10100010	1, 3, 7	$3 + 4 + 2 = 9$	14
11000000	1, 2	$3 + 7 = 10$	0
00010010	4, 7	$1 + 2 = 3$	6

Discusión

¿Una penalización igual a cero diferencia adecuadamente entre exceder la capacidad por 1 kg y por 20 kg?

4. Selección de padres

La selección favorece la aptitud, pero no debe eliminar toda posibilidad para individuos distintos.

Ruleta

$$p_i = \frac{f_i}{\sum_j f_j}$$

Una mayor aptitud produce mayor probabilidad de selección.

Alternativas

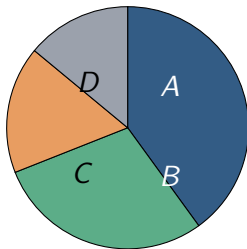
- ▶ torneo;
- ▶ selección por rango;
- ▶ elitismo para conservar los mejores.

Presión selectiva

Demasiada presión acelera la convergencia, pero puede reducir prematuramente la diversidad.

Ejemplo de selección por ruleta

Ind.	Aptitud	Prob.
<i>A</i>	14	0.40
<i>B</i>	10	0.29
<i>C</i>	6	0.17
<i>D</i>	5	0.14



A es más probable, pero *C* y *D* aún pueden aportar genes útiles.

5. Cruce de un punto

Padre A: 1 0 1 0 | 0 0 1 0

Padre B: 0 1 0 1 | 1 0 0 0

Hijo 1: 1 0 1 0 | 1 0 0 0

Hijo 2: 0 1 0 1 | 0 0 1 0

El punto de corte determina qué segmentos se heredan de cada progenitor.

Otras estrategias de cruce

Un punto

Dos segmentos. Simple y económico.

Dos puntos

Tres segmentos alternados. Mezcla intermedia.

Uniforme

Una máscara decide gen a gen. Mayor mezcla.

Más mezcla $\nrightarrow$ mejores descendientes.

La estrategia debe respetar la semántica de la codificación.

6. Mutación

En codificación binaria, la mutación de bit cambia $0 \leftrightarrow 1$ con una probabilidad pequeña.

1 0 1 0 1 0 0 0



1 0 1 1 1 0 0 0

Función

- ▶ introducir novedad;
- ▶ recuperar diversidad;
- ▶ escapar de óptimos locales.

Tasa de mutación

Muy baja: estancamiento. Muy alta: la búsqueda se aproxima a un proceso aleatorio.

7. Reemplazo de la población

Modelo generacional

La descendencia reemplaza toda la población.

- ▶ Mayor renovación.
- ▶ Riesgo de perder buenos individuos.

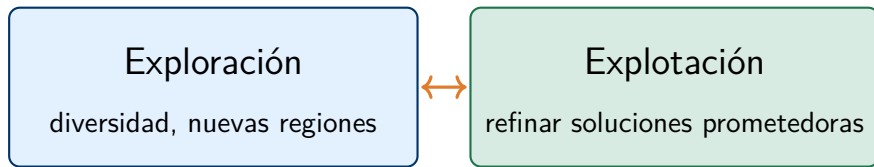
Estado estable

Solo una parte de la población es reemplazada.

- ▶ Cambio gradual.
- ▶ Conserva más información previa.

El **elitismo** copia uno o más individuos destacados a la siguiente generación.

Exploración y explotación



- ▶ Población diversa y mutación favorecen exploración.
- ▶ Selección intensa y elitismo favorecen explotación.
- ▶ La configuración adecuada depende del problema y puede variar con el tiempo.

Condiciones de parada

- ▶ Número máximo de generaciones.
- ▶ Aptitud objetivo alcanzada.
- ▶ Presupuesto de tiempo o evaluaciones agotado.
- ▶ Ausencia de mejora durante varias generaciones.
- ▶ Pérdida excesiva de diversidad.

Salida

Se devuelve el individuo con mejor aptitud observado, junto con sus métricas y restricciones verificadas.

Pseudocódigo general

```
P <- inicializar_poblacion()
evaluar(P)

mientras no se cumpla la condicion_de_parada:
    padres <- seleccionar(P)
    hijos <- cruzar(padres)
    hijos <- mutar(hijos)
    evaluar(hijos)
    P <- reemplazar(P, hijos)

devolver mejor_individuo(P)
```

Detalle clave

Registrar el mejor individuo de toda la ejecución evita perder una buena solución por reemplazo o mutación.

Parámetros que controlan el comportamiento

Parámetro	Si aumenta	Posible costo
Población	más diversidad	más evaluaciones
Descendencia	más alternativas	reemplazo más agresivo
Mutación	más exploración	deterioro de buenas soluciones
Presión selectiva	convergencia rápida	pérdida de diversidad
Generaciones	mayor oportunidad de mejora	más tiempo

Complejidad aproximada

Si N es el tamaño de la población, G las generaciones y C_f el costo de evaluar un individuo:

$$T \approx O(G N C_f)$$

Para un cromosoma de longitud L , una aptitud que recorre todos los genes suele tener $C_f = O(L)$:

$$T \approx O(GNL)$$

Lectura correcta

El algoritmo no examina todo el espacio 2^L ; examina aproximadamente $G \times N$ candidatos, aunque algunos pueden repetirse.

Errores frecuentes de diseño

- ❶ Codificación que genera demasiadas soluciones inválidas.
- ❷ Aptitud que no representa el objetivo real.
- ❸ Eliminar toda diversidad al seleccionar solo los mejores.
- ❹ Usar una mutación excesiva o casi inexistente.
- ❺ Evaluar una sola ejecución de un algoritmo estocástico.
- ❻ Comparar métodos con presupuestos computacionales distintos.

¿Cómo evaluar experimentalmente el algoritmo?

Calidad de solución

- ▶ mejor aptitud;
- ▶ promedio y desviación;
- ▶ distancia al óptimo conocido;
- ▶ cumplimiento de restricciones.

Costo y estabilidad

- ▶ tiempo y memoria;
- ▶ número de evaluaciones;
- ▶ convergencia por generación;
- ▶ varias semillas aleatorias.

Una sola ejecución no basta para caracterizar un método estocástico.

- ▶ selección de variables en aprendizaje automático;
- ▶ planificación de rutas, horarios y asignación de recursos;
- ▶ diseño y ajuste de redes y sistemas de comunicación;
- ▶ ajuste de hiperparámetros de modelos;
- ▶ diseño de estructuras, circuitos y controladores;
- ▶ evolución de reglas, expresiones y pequeños programas.

Antes de elegirlo

Comprobar si existe un método exacto, heurístico o específico del dominio que sea más simple y confiable.

Actividad en equipos: diseñar un algoritmo genético

Problema: seleccionar proyectos de IA con presupuesto limitado.

- ① Definir el cromosoma y el significado de cada gen.
- ② Proponer una función de aptitud con costo y beneficio.
- ③ Elegir selección, cruce y mutación.
- ④ Decidir cómo tratar soluciones que excedan el presupuesto.
- ⑤ Fijar una condición de parada y dos métricas de evaluación.

Producto

Un diagrama del ciclo y un ejemplo manual de una generación con cuatro individuos.

- ① ¿Por qué la diversidad es importante al inicio?
- ② ¿Qué diferencia existe entre aptitud y probabilidad de selección?
- ③ ¿Cómo afecta una tasa de mutación muy alta?
- ④ ¿Cuándo una solución “suficientemente buena” no sería aceptable?
- ⑤ ¿Por qué conviene ejecutar el algoritmo varias veces?

- ▶ El algoritmo genético busca con una **población**, no con una única trayectoria.
- ▶ La **representación** y la **aptitud** determinan qué puede aprender la búsqueda.
- ▶ Selección, cruce y mutación equilibran **explotación** y **exploración**.
- ▶ El resultado es estocástico: debe evaluarse con **múltiples ejecuciones**.
- ▶ Una solución de alta calidad no implica una garantía de óptimo global.

Fuente principal

Capítulo 4, *Evolutionary algorithms*, documento proporcionado para la sesión, pp. 91–129.

- ▶ Revisar el ciclo de vida del algoritmo genético.
- ▶ Repasar el ejemplo de la mochila y sus restricciones.
- ▶ Comparar cruce de uno y dos puntos con cruce uniforme.
- ▶ Identificar los parámetros que afectan diversidad y convergencia.

Preguntas y discusión

¿Qué problema de su contexto podría representarse como una población de soluciones?